Resume Screening and Campus Placement Management API

Project Proposal for UCS503P
Submitted to: Paramveer Kaur
Thapar Institute of Engineering and Technology

Kanika Sahni
Roll No: 1024030031

Navya Khosla
Roll No: 1024030029

Abhinandan Wadhwa
Roll No: 1024030026

August 16, 2026

1 Higher-order goal

. . . secondary framing

This project aims to streamline campus placement operations by connecting student profile creation, resume-based data extraction, recruiter job management, candidate discovery, applications, and placement-office administration in one web-based system. The engineering objective is to reduce manual data entry and scattered recruitment workflows while keeping candidate information searchable, persistent, and usable by different stakeholders.

To achieve this, the proposed architecture will utilize a FastAPI backend for robust API routing, dedicated HTML/CSS interfaces tailored for student, recruiter, and administrator roles, and a managed PostgreSQL-backed data layer (such as Supabase) for secure and relational data persistence.

2 Time-to-value

. . . primary heuristic

- The proposed system will provide immediate value at student onboarding by accepting a PDF resume and extracting a draft skills list and short text preview.
- Recruiters will be able to move directly from role creation to candidate search and applicant review without maintaining separate spreadsheets or manually collecting profiles.
- The placement office will gain a centralized view of student placement status, recruiter activity, recruitment statistics, and recent audit activity.
- Incremental delivery will be practical because the application will be exposed as REST-style API endpoints consumed by lightweight web pages.

3 Problem Statement

Campus placement processes often distribute student data, job openings, applications, and recruiter communication across unrelated forms, spreadsheets, and portals. This creates several operational problems:

- **Manual profile entry:** students repeatedly enter skills and summary information that already exists in their resumes.

- **Fragmented candidate discovery:** recruiters need a searchable view of active candidates rather than manually scanning every student profile.
- **Disconnected application tracking:** job postings and student applications need a common source of truth.
- **Limited administrative visibility:** placement teams need current counts, recruiter activity, and student placement status in one dashboard.
- **Data consistency:** skills should be stored in a normalized way so that multiple students can be searched and matched against the same skill vocabulary.

4 Proposed Solution *. . . Software Proposition*

4.1 Overview

We propose to build a **web-based campus placement platform** with resume-assisted profile creation and role-specific workflows:

- **Resume parsing API:** accept a PDF, extract text with PyMuPDF, scan for a predefined set of technical keywords, and return a suggested skills string plus a short bio preview.
- **Student portal:** allow students to register, log in, view profile information, edit CGPA and skills, browse jobs, and submit applications.
- **Recruiter portal:** enable recruiters to authenticate with corporate credentials, publish jobs, view applicants for their roles, export applicant data, and search active candidates by skills.
- **Placement administration:** provide tools to manage recruiter accounts, inspect student records, mark students as placed, monitor recruitment statistics, and view recent audit logs.
- **Relational data layer:** persist students, skills, student-skill relationships, jobs, applications, recruiters, and placement/audit information in PostgreSQL.

4.2 Core workflow

1. A student will upload a PDF resume during registration.
2. The backend will extract PDF text and check it against a predefined keyword list such as Python, C++, Java, SQL, React, ML, AI, DSA, and JavaScript.
3. The student will review the extracted suggestions, enter or confirm profile details such as CGPA, GitHub, skills, and professional summary, and create the account.
4. Recruiters will log in, create job postings containing role, company, salary/CTC, experience, location, and description, and make those jobs available to students.
5. A student will submit an application to a job. The backend will prevent duplicate applications where the underlying database constraint reports an existing duplicate.
6. Recruiters will retrieve applicants for their jobs and search active candidates using a database-backed skill-match score, ordered with CGPA as a secondary criterion.
7. Placement administrators will monitor student placement records, recruitment statistics, recruiter activity, and audit events.

4.3 Operational constraints for an educational placement setting

- Keep the web interfaces usable on common desktop and laptop browsers with responsive layouts.
- Keep the resume-processing step lightweight and deterministic; the implementation will be keyword-based rather than a learned resume-classification model.
- Use a managed PostgreSQL-compatible database for persistent storage while exposing the application through HTTP APIs.
- Separate the three stakeholder experiences (student, recruiter, administrator) so that each role sees only the workflow relevant to it at the interface level.

5 Solution Approach *... Engineering focus*

The project will primarily be a full-stack software engineering and information-management solution. Its value will come from integrating resume text extraction, profile storage, candidate search, job/application workflows, and administrative analytics rather than from introducing a novel machine-learning algorithm.

5.1 Resume processing and skill extraction *... implementation-grounded*

Building upon the base repository's `/ai_parse` endpoint, the system will read an uploaded PDF using PyMuPDF (the `fitz` package), concatenate page text, and perform case-insensitive substring matching against a fixed list of technical keywords. The endpoint will return `suggested_skills` and the first 300 characters as `bio_preview`.

This will be implemented as **rule-based keyword extraction from PDF text**, not as a trained artificial-intelligence model. A future enhancement could replace the fixed list with NER, embedding-based semantic matching, or a trained classifier, but those techniques are outside the planned initial scope.

5.2 Web architecture *... API-centered*

The planned three-layer architecture includes:

- **Frontend:** HTML/CSS/JavaScript pages for the landing page, student registration/workspace, recruiter workspace, login pages, and placement administration.
- **Backend API:** FastAPI endpoints for resume parsing, student registration/login/update, candidate search, job creation, applications, recruiter onboarding/login, recruiter applicant retrieval, placement updates, statistics, and audit logs.
- **Database:** PostgreSQL accessed with `psycopg2`; the schema will include `students`, `skills`, and `student_skills`, while other endpoints will operate over recruiter, job, application, statistics, candidate-view, and audit-log structures.

5.3 API surface

The proposed backend will expose role-specific routes including:

- `POST /ai_parse` for resume PDF parsing.
- `POST /register_student`, `POST /login`, and `POST /update_profile` for student profile lifecycle management.

- `GET /recruiter/candidates` for candidate search and match scoring.
- `POST /recruiter/jobs` and `GET /jobs` for role publishing and job retrieval.
- `POST /student/apply/{job_id}` for applications.
- `GET /recruiter/applicants` for recruiter-specific application review.
- `POST /admin/add_recruiter`, `POST /admin/mark_placed/{student_id}`, `GET /admin/stats`, `GET /admin/all_students`, and `GET /admin/audit_logs` for placement-office operations.

5.4 Authentication and data handling

Student passwords will be hashed using Passlib with bcrypt before storage, while student logins will verify the supplied password against the stored hash. Recruiter authentication will use an email/access-code pair. The backend will also remove the stored student password from the successful student-login response before returning the user object.

For deployment, credentials, database connection strings, administrator secrets, and recruiter access codes will be moved to environment-managed secrets rather than being embedded in source code (which is a vulnerability noted in the baseline repository).

6 Evaluation Criterion

... measurable and attributable

The platform will be evaluated using operational metrics tied to its principal workflows.

6.1 Primary evaluation metric

Profile onboarding time: time from the beginning of student registration to creation of a complete student profile.

- Target: reduce average onboarding time compared with fully manual profile entry.
- Attribution: measure from the registration workflow and successful profile creation timestamps where available.

6.2 Secondary evaluation metrics

- **Resume extraction usefulness:** percentage of uploaded resumes for which the keyword extractor returns at least one relevant skill.
- **Candidate-search relevance:** proportion of top-ranked search results judged relevant to the recruiter-entered skill set.
- **Application processing efficiency:** time required for recruiters to locate and inspect applicants for a role compared with a manual process.
- **Data completeness:** percentage of student profiles containing CGPA, skills, GitHub profile, and professional summary.
- **Duplicate-application prevention:** percentage of repeated submissions correctly blocked by the application constraint/error handling.
- **Administrative visibility:** availability of current student, placement, recruiter, and audit information from the administrative dashboard.

6.3 Validation plan *... practical pilot*

- Use a sample of student resumes representing different formatting styles and technology stacks.
- Ask student testers to register through the complete upload-to-profile workflow and record time, extraction usefulness, and correction effort.
- Ask recruiter testers to publish a role, search candidates by several skill combinations, inspect applicants, and export applicant data.
- Ask placement-office testers to onboard recruiters, inspect statistics, mark a student as placed, and review audit activity.

7 Scalability *... with foresight*

While the planned API will provide a solid foundation, several improvements will be scoped for scalable deployment:

1. **Scalable (theoretically):** support growth in candidate and application volume through:
   - database indexing on commonly searched fields,
   - pagination for student, job, and applicant queries,
   - connection pooling and resilient database handling,
   - stateless API instances behind a load balancer, and
   - caching for frequently requested public job data and aggregate statistics.
2. **Search quality (practically):** evolve from fixed keyword matching toward normalized skill taxonomies and semantic matching so that related skills and spelling variants can be handled consistently.
3. **Operational deployment:** separate frontend and backend deployment, use managed PostgreSQL, keep secrets outside source control, and add automated testing and deployment checks.

8 Engine availability heuristic

The studied repository demonstrates that a mature-enough stack exists to successfully build this system for a student-scale deployment. We will utilize:

- **FastAPI + Uvicorn:** HTTP API layer and development/application server.
- **PyMuPDF:** PDF text extraction for resume uploads.
- **Passlib + bcrypt:** password hashing and verification for student accounts.
- **PostgreSQL + psycopg2:** relational persistence and SQL access.
- **HTML/CSS/JavaScript:** lightweight stakeholder-facing web interfaces.
- **Vercel/Render-style deployment:** hosting configuration for continuous delivery.

9 Project scope and deliverables *... iteration-friendly*

9.1 Initial deliverable *... first iteration*

- Student registration and login with relational profile storage.
- PDF resume upload with rule-based skill extraction and bio preview.
- Student profile editing for CGPA and skills.
- Recruiter login and role creation.
- Public/Student job listing and job application submission.
- Recruiter applicant inbox and skill-based candidate search.
- Placement-office dashboard with student records and placement status.

9.2 Subsequent deliverables

- Replace fixed keyword extraction with normalized skill matching or semantic NLP.
- Add explicit application-status transitions such as shortlisted, interview, selected, and rejected.
- Add stronger role-based authorization and session/token management.
- Add automated unit/integration tests and CI/CD quality gates.
- Improve audit coverage so important recruiter and student changes are traceable.
- Add analytics for conversion rates, source of applicants, skill demand, and placement outcomes.

10 Risks and mitigations

- **Credential exposure:** We will migrate all database credentials, administrator secrets, and access codes to environment-managed secrets to avoid hardcoding them into source control.
- **Weak resume interpretation:** Fixed substring matching can miss synonyms, variants, and contextual meaning; we will introduce normalization and semantic models in a later iteration.
- **Authorization gaps:** We will replace simple credential checks with robust role-based authorization and expiring sessions/tokens.
- **Data quality:** We will validate profile fields and normalize skills to reduce duplicates and inconsistent spellings.
- **Scale and reliability:** We will add pagination, indexes, connection pooling, monitoring, and automated tests before handling a large placement cohort.

11 Summary

This project proposes to build a deployable campus placement platform that combines resume-assisted student onboarding with recruiter and placement-office workflows. The proposed implementation will be grounded in FastAPI, PostgreSQL, PyMuPDF, bcrypt-based password hashing, and role-specific web interfaces. While the initial resume feature will act as a lightweight keyword-extraction mechanism rather than a learned AI screener, its broader engineering value will come from successfully integrating candidate search, job posting, applications, recruiter management, placement statistics, and audit activity into a single unified system.